Aws First Challenge

Launching a Single EC2 Instance Running Docker, Jenkins, Apache, Nginx, and Apache Tomcat

Assignment Overview

This document provides complete step-by-step instructions to launch a single Amazon EC2 instance running Amazon Linux 2, and to install and run Docker, Jenkins, Apache (httpd), Nginx, and Apache Tomcat, all on that same instance simultaneously.

Because Apache, Jenkins, Nginx, and Tomcat all default to well-known ports (80 or 8080), the following non-conflicting port plan is used throughout this document:

Service	Port	Access URL
Apache (httpd)	80	http://<Public-IP>
Jenkins	8080	http://<Public-IP>:8080
Nginx	8081 (changed from default 80)	http://<Public-IP>:8081
Apache Tomcat	8082 (changed from default 8080)	http://<Public-IP>:8082
Docker	N/A (CLI / daemon only)	docker ps on the instance
SSH	22	ssh ec2-user@<Public-IP>

Task 1: Launch One EC2 Instance Using Amazon Linux 2

Outcome

A single EC2 instance named “FirstChallenge” is launched using the Amazon Linux 2 AMI, reaches the “Running” state with 2/2 status checks passed, and is reachable over SSH. This instance will host Docker, Jenkins, Apache, Nginx, and Tomcat together.

Objective

To provision a single virtual server in AWS that acts as the common host for multiple DevOps tools, simulating a lightweight all-in-one environment for CI/CD and web-hosting practice.

Prerequisites

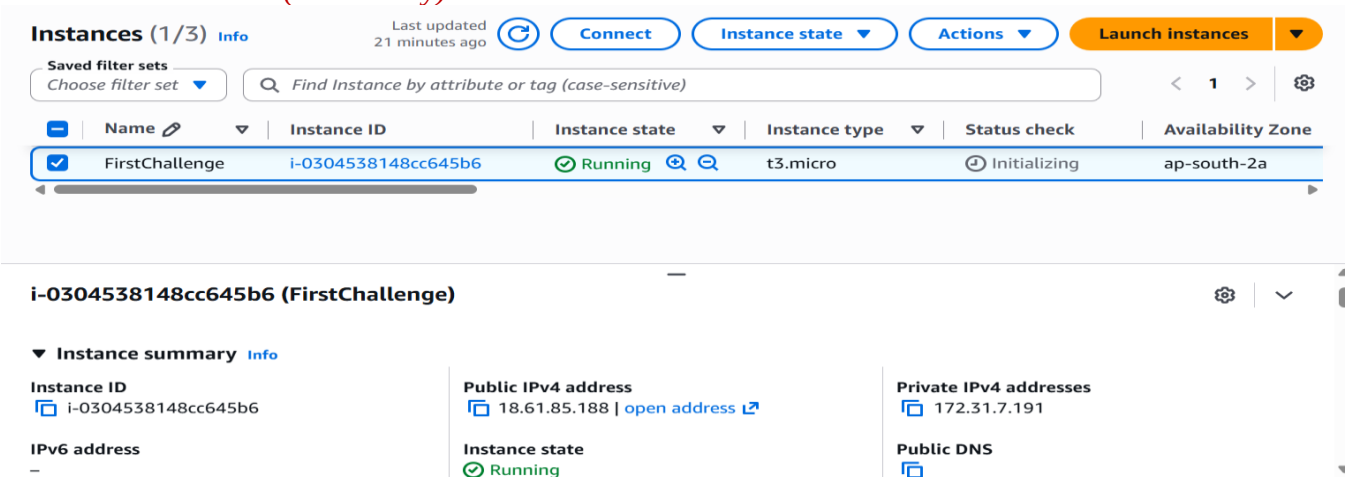
- AWS account with console access.
- IAM permissions to create EC2 instances, key pairs, and security groups.
- A region selected (e.g., ap-south-1 or us-east-1).
- An SSH client (Terminal, PuTTY, or EC2 Instance Connect) available locally.

Step-by-Step Implementation

Step 1: Open the EC2 Console

Login to AWS Console → Search for “EC2” → Open EC2 Dashboard → Click “Instances” in the left panel.

***Screenshot Evidence** (Mandatory)*



Step 2: Launch Instance and Choose AMI

Click “Launch instance” and configure the basic settings:

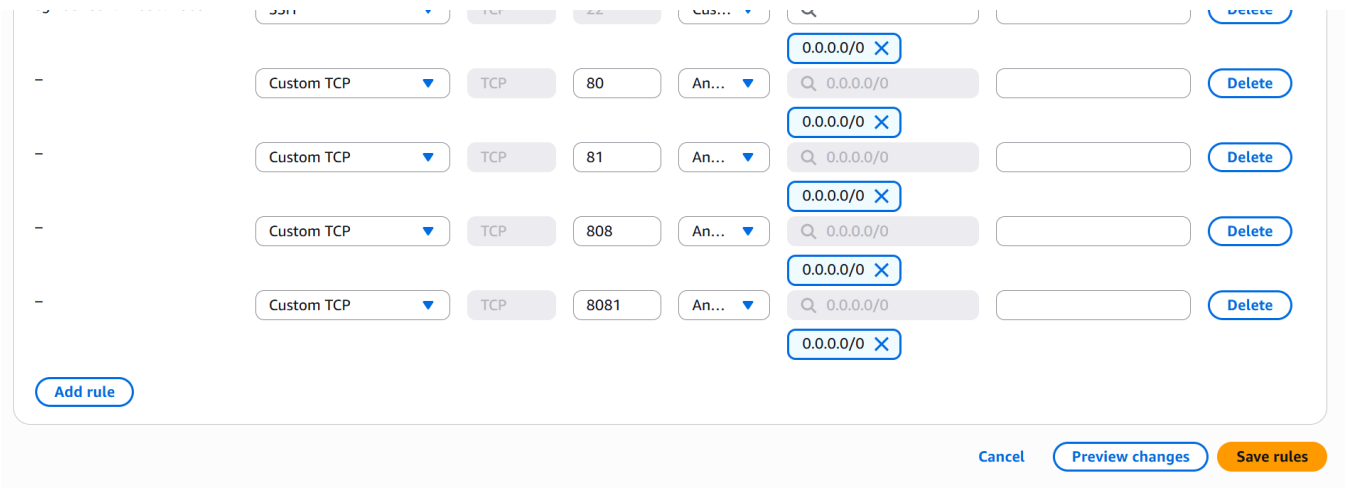
Parameter	Value
Name	FirstChallenge
Amazon Machine Image (AMI)	Amazon Linux 2 AMI (HVM), SSD Volume Type
Instance type	t3.medium (recommended; t2.micro is Free-Tier eligible but tight on RAM for Jenkins + Tomcat + Docker together)
Key pair	Create new key pair → Name: multitool-key → Type: RSA → Format: challenge.pem

Step 3: Configure Network Settings and Security Group

Under “Network settings”, keep the default VPC and enable “Auto-assign public IP”. Create a new security group named MultiTool-SG with the following inbound rules:

Type	Protocol	Port Range	Source
SSH	TCP	22	My IP
HTTP (Apache)	TCP	80	Anywhere (0.0.0.0/0)
Custom TCP (Jenkins)	TCP	8080	Anywhere (0.0.0.0/0)
Custom TCP (Nginx)	TCP	8081	Anywhere (0.0.0.0/0)
Custom TCP (Tomcat)	TCP	8082	Anywhere (0.0.0.0/0)

***Screenshot Evidence** (Mandatory) —*



Step 4: Launch and Connect

Click “Launch instance” → wait until the instance state shows “Running” → copy the Public IPv4 address → connect using SSH:

```
chmod 400 challenge.pem
ssh -i challenge.pem ec2-user@<Public-IP>
```

***Screenshot Evidence** (Mandatory) —*

Validation Steps

- EC2 console shows FirstChallenge with Instance state “Running” and Status check “2/2 checks passed”.
- SSH session connects successfully and the shell prompt shows [ec2-user@ip-xxx-xxx-xxx-xxx ~]$.

Explanation

An EC2 instance is a virtual machine running in AWS’s data centers that provides the compute, memory, and network resources required to run software, exactly like a physical server. Amazon Linux 2 was chosen because it is optimized for AWS (fast boot, tuned kernel), is Free-Tier eligible, ships with yum and amazon-linux-extras for simple package management, and is a long-term-support distribution commonly used in production. This instance is important because it forms the single shared host on which every subsequent tool in this assignment — Docker, Jenkins, Apache, Nginx, and Tomcat — will be installed and must coexist without port conflicts.

Real-Time Use Case

Organizations commonly provision a base EC2 instance as the starting point for a build server, a container host, or a small internal web server. Running multiple lightweight services (a CI server, a couple of web servers, and an application server) on one instance mirrors how small teams build a proof-of-concept DevOps sandbox or training lab before scaling out to dedicated instances, auto-scaling groups, or a container orchestration platform such as ECS or Kubernetes.

Common Errors and Troubleshooting

Error	Cause	Resolution
“Connection timed out” during SSH	Security group missing inbound rule for port 22, or connecting from a different network/IP than allowed	Edit the security group to allow port 22 from “My IP” (or the current IP) and retry
“Permission denied (publickey)”	Wrong username used, or wrong .pem file, or key file permissions too open	Use username ec2-user for Amazon Linux 2; run chmod 400 key.pem; confirm the key pair matches the one selected at launch

Error	Cause	Resolution
Instance stuck in “pending” for a long time	Temporary capacity issue in the Availability Zone for the chosen instance type	Wait and retry, or relaunch choosing a different instance type or Availability Zone

Task 2: Install Docker

Outcome

Docker Engine is installed on the EC2 instance, the docker service is active and enabled to start on boot, and running docker run hello-world completes successfully.

Objective

To provide a containerization platform on the instance so that containerized applications, Jenkins build agents, or microservices can be built and run in isolated environments.

Prerequisites

- Task 1 completed — EC2 instance running Amazon Linux 2 and reachable via SSH.
- sudo privileges on the instance (ec2-user has sudo by default).
- Outbound internet access from the instance to reach the yum repositories.

Step-by-Step Implementation

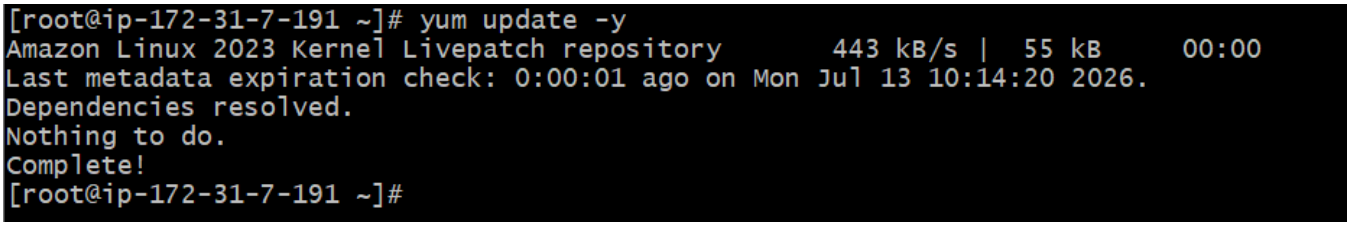
Step 1: Connect to the Instance

```
ssh -i Challenge.pem ec2-user@<Public-IP>
```

Step 2: Update Installed Packages

```
 yum update -y
```

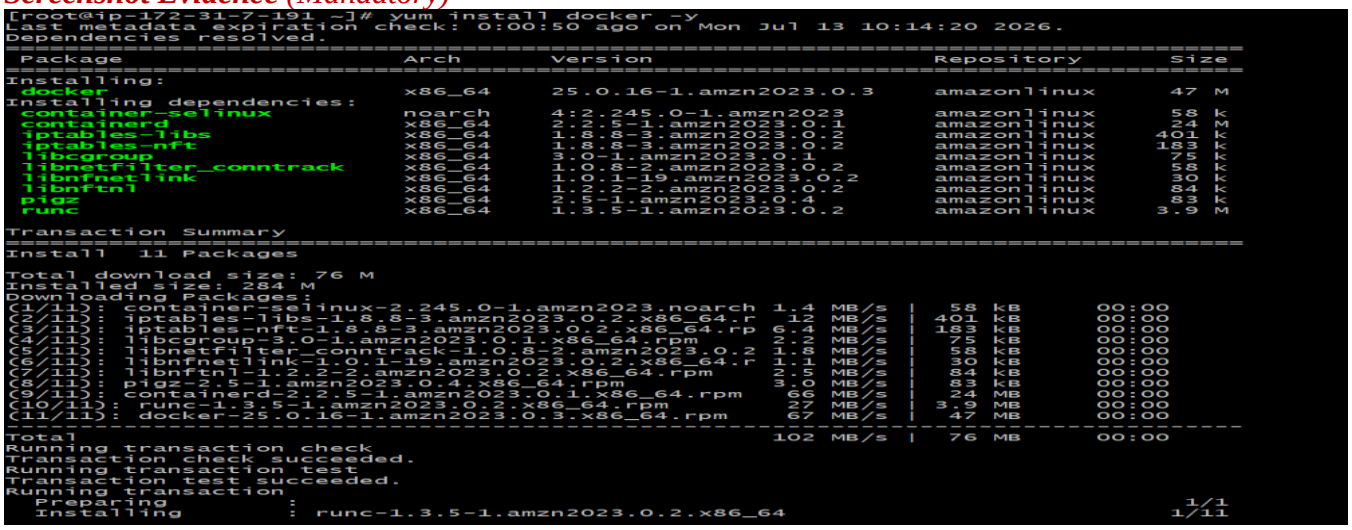
***Screenshot Evidence** (Mandatory) —*



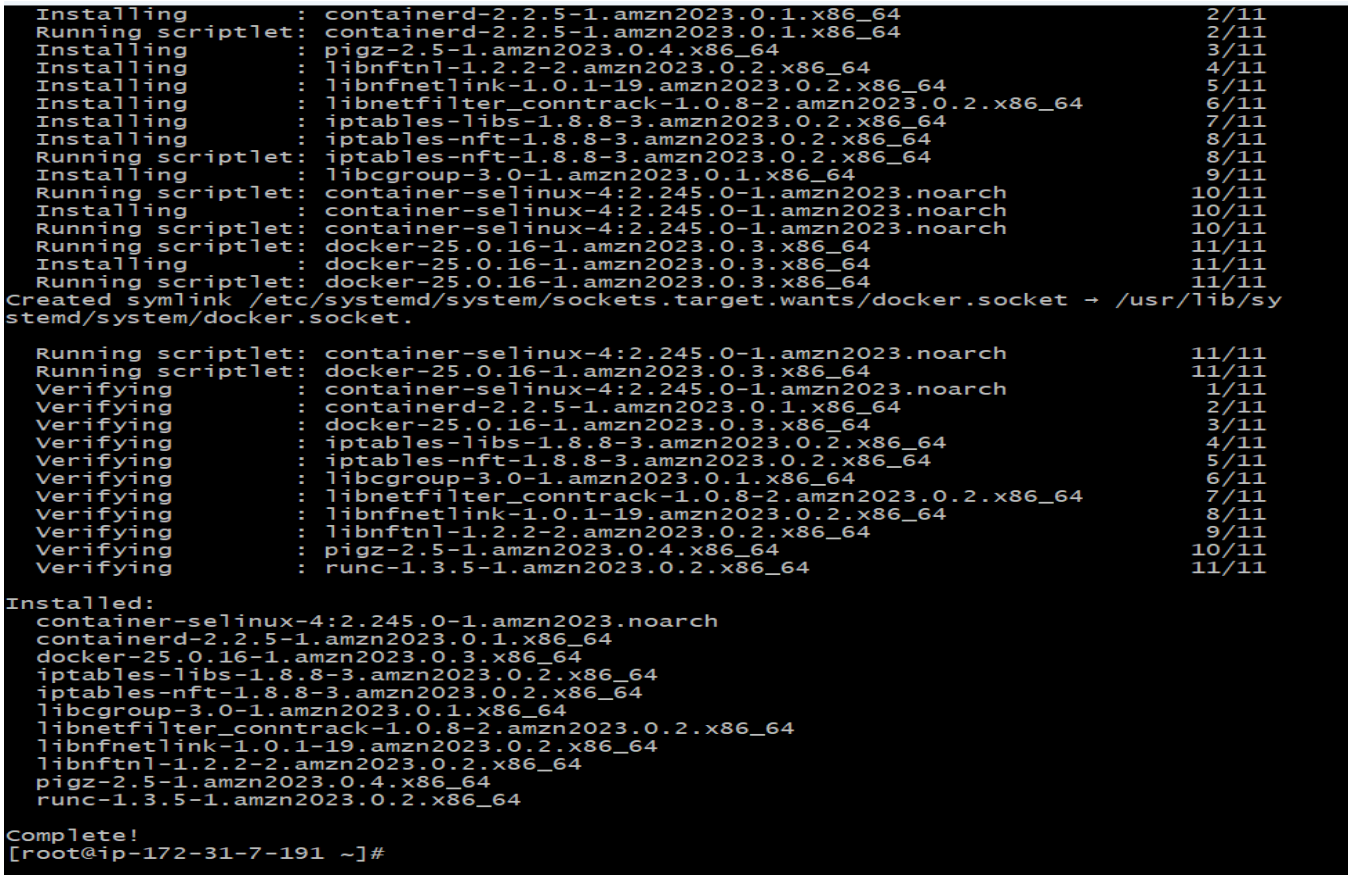
Step 3: Install Docker

```
 yum install docker -y
```

***Screenshot Evidence** (Mandatory) —*



```
[root@ip-172-31-7-191 ~]# yum install docker -y
Last metadata expiration check: 0:00:50 ago on Mon Jul 13 10:14:20 2026.
Dependencies resolved.
================================================================================
 Package                  Arch       Version                    Repository    Size
================================================================================
Installing:
 docker                   x86_64     25.0.16-1.amzn2023.0.3     amazonlinux   47 M
Installing dependencies:
 container-selinux        noarch     4:2.245.0-1.amzn2023       amazonlinux   58 k
 containerd               x86_64     2.2.5-1.amzn2023.0.1       amazonlinux   24 M
 iptables-libs            x86_64     1.8.8-3.amzn2023.0.2       amazonlinux  401 k
 iptables-nft             x86_64     1.8.8-3.amzn2023.0.2       amazonlinux  183 k
 libcgroup                x86_64     3.0-1.amzn2023.0.1         amazonlinux   75 k
 libnetfilter_conntrack   x86_64     1.0.8-2.amzn2023.0.2       amazonlinux   58 k
 libnfnetlink             x86_64     1.0.1-19.amzn2023.0.2      amazonlinux   30 k
 libnftnl                 x86_64     1.2.2-2.amzn2023.0.2       amazonlinux   84 k
 pigz                     x86_64     2.5-1.amzn2023.0.4         amazonlinux   83 k
 runc                     x86_64     1.3.5-1.amzn2023.0.2       amazonlinux  3.9 M

Transaction Summary
================================================================================
Install  11 Packages

Total download size: 76 M
Installed size: 284 M
Downloading Packages:
(1/11): container-selinux-2.245.0-1.amzn2023.noarch 1.4 MB/s |  58 kB     00:00
(2/11): iptables-libs-1.8.8-3.amzn2023.0.2.x86_64.r  12 MB/s | 401 kB     00:00
(3/11): iptables-nft-1.8.8-3.amzn2023.0.2.x86_64.rp 6.4 MB/s | 183 kB     00:00
(4/11): libcgroup-3.0-1.amzn2023.0.1.x86_64.rpm     2.2 MB/s |  75 kB     00:00
(5/11): libnetfilter_conntrack-1.0.8-2.amzn2023.0.2 1.8 MB/s |  58 kB     00:00
(6/11): libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64.r 1.1 MB/s |  30 kB     00:00
(7/11): libnftnl-1.2.2-2.amzn2023.0.2.x86_64.rpm    2.5 MB/s |  84 kB     00:00
(8/11): pigz-2.5-1.amzn2023.0.4.x86_64.rpm          3.0 MB/s |  83 kB     00:00
(9/11): containerd-2.2.5-1.amzn2023.0.1.x86_64.rpm   66 MB/s |  24 MB     00:00
(10/11): runc-1.3.5-1.amzn2023.0.2.x86_64.rpm        27 MB/s | 3.9 MB     00:00
(11/11): docker-25.0.16-1.amzn2023.0.3.x86_64.rpm    67 MB/s |  47 MB     00:00
--------------------------------------------------------------------------------
Total                                               102 MB/s |  76 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : runc-1.3.5-1.amzn2023.0.2.x86_64                      1/11
```



```
  Installing       : containerd-2.2.5-1.amzn2023.0.1.x86_64                 2/11
  Running scriptlet: containerd-2.2.5-1.amzn2023.0.1.x86_64                 2/11
  Installing       : pigz-2.5-1.amzn2023.0.4.x86_64                         3/11
  Installing       : libnftnl-1.2.2-2.amzn2023.0.2.x86_64                   4/11
  Installing       : libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64              5/11
  Installing       : libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64     6/11
  Installing       : iptables-libs-1.8.8-3.amzn2023.0.2.x86_64              7/11
  Installing       : iptables-nft-1.8.8-3.amzn2023.0.2.x86_64               8/11
  Running scriptlet: iptables-nft-1.8.8-3.amzn2023.0.2.x86_64               8/11
  Installing       : libcgroup-3.0-1.amzn2023.0.1.x86_64                    9/11
  Running scriptlet: container-selinux-4:2.245.0-1.amzn2023.noarch         10/11
  Installing       : container-selinux-4:2.245.0-1.amzn2023.noarch         10/11
  Running scriptlet: container-selinux-4:2.245.0-1.amzn2023.noarch         10/11
  Running scriptlet: docker-25.0.16-1.amzn2023.0.3.x86_64                  11/11
  Installing       : docker-25.0.16-1.amzn2023.0.3.x86_64                  11/11
  Running scriptlet: docker-25.0.16-1.amzn2023.0.3.x86_64                  11/11
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/sy
stemd/system/docker.socket.

  Running scriptlet: container-selinux-4:2.245.0-1.amzn2023.noarch         11/11
  Running scriptlet: docker-25.0.16-1.amzn2023.0.3.x86_64                  11/11
  Verifying        : container-selinux-4:2.245.0-1.amzn2023.noarch          1/11
  Verifying        : containerd-2.2.5-1.amzn2023.0.1.x86_64                 2/11
  Verifying        : docker-25.0.16-1.amzn2023.0.3.x86_64                   3/11
  Verifying        : iptables-libs-1.8.8-3.amzn2023.0.2.x86_64              4/11
  Verifying        : iptables-nft-1.8.8-3.amzn2023.0.2.x86_64               5/11
  Verifying        : libcgroup-3.0-1.amzn2023.0.1.x86_64                    6/11
  Verifying        : libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64     7/11
  Verifying        : libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64              8/11
  Verifying        : libnftnl-1.2.2-2.amzn2023.0.2.x86_64                   9/11
  Verifying        : pigz-2.5-1.amzn2023.0.4.x86_64                        10/11
  Verifying        : runc-1.3.5-1.amzn2023.0.2.x86_64                      11/11

Installed:
  container-selinux-4:2.245.0-1.amzn2023.noarch
  containerd-2.2.5-1.amzn2023.0.1.x86_64
  docker-25.0.16-1.amzn2023.0.3.x86_64
  iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
  iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
  libcgroup-3.0-1.amzn2023.0.1.x86_64
  libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
  libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64
  libnftnl-1.2.2-2.amzn2023.0.2.x86_64
  pigz-2.5-1.amzn2023.0.4.x86_64
  runc-1.3.5-1.amzn2023.0.2.x86_64

Complete!
[root@ip-172-31-7-191 ~]#
```

Step 4: Start and Enable the Docker Service

```
systemctl start docker
systemctl enable docker

systemctl status docker
```

***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 ~]# systemctl start docker
[root@ip-172-31-7-191 ~]# systemctl enable docker
-bash: systemctl: command not found
[root@ip-172-31-7-191 ~]# systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/li
b/systemd/system/docker.service.
[root@ip-172-31-7-191 ~]# systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disab>
     Active: active (running) since Mon 2026-07-13 10:17:07 UTC; 1min 17s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 28607 (dockerd)
      Tasks: 9
     Memory: 30.5M
        CPU: 356ms
     CGroup: /system.slice/docker.service
             └─28607 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/contain>

Jul 13 10:17:06 ip-172-31-7-191.ap-south-2.compute.internal systemd[1]: Starting do>
Jul 13 10:17:06 ip-172-31-7-191.ap-south-2.compute.internal dockerd[28607]: time="2>
Jul 13 10:17:06 ip-172-31-7-191.ap-south-2.compute.internal dockerd[28607]: time="2>
Jul 13 10:17:07 ip-172-31-7-191.ap-south-2.compute.internal dockerd[28607]: time="2>
Jul 13 10:17:07 ip-172-31-7-191.ap-south-2.compute.internal dockerd[28607]: time="2>
Jul 13 10:17:07 ip-172-31-7-191.ap-south-2.compute.internal dockerd[28607]: time="2>
Jul 13 10:17:07 ip-172-31-7-191.ap-south-2.compute.internal dockerd[28607]: time="2>
Jul 13 10:17:07 ip-172-31-7-191.ap-south-2.compute.internal systemd[1]: Started doc>
lines 1-20/20 (END)...skipping...
```

Step 5: Verify the Installation

```
docker --version
docker run hello-world
```

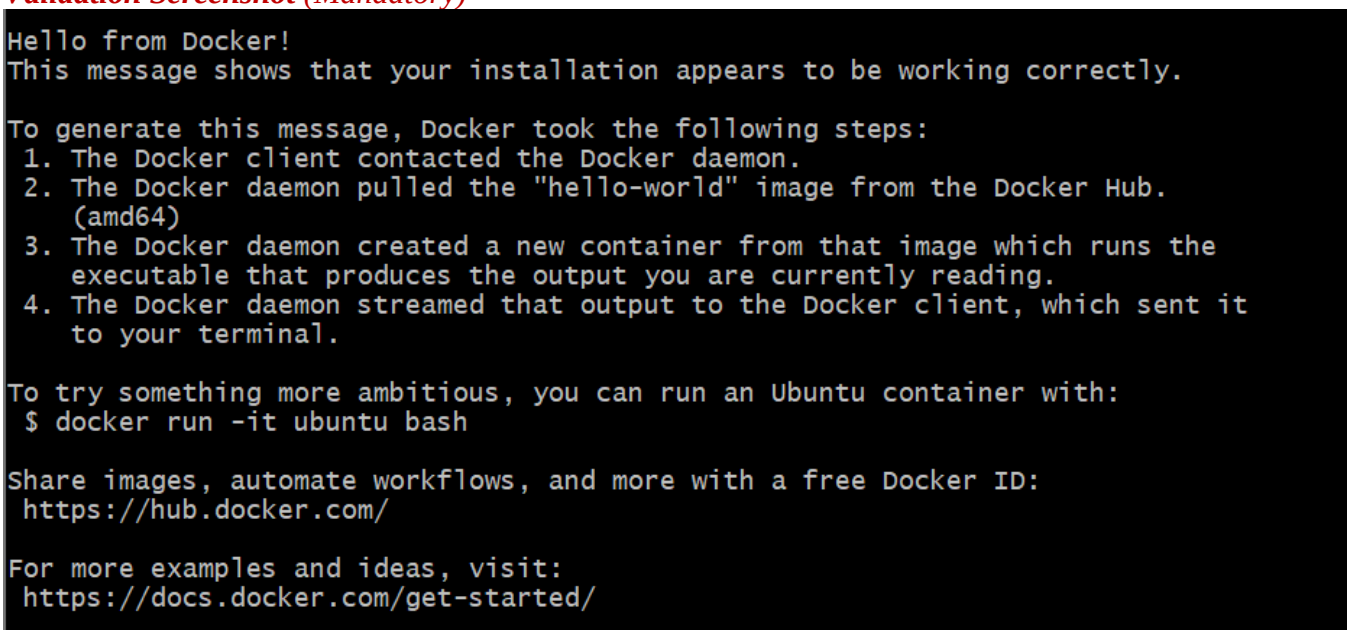
***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 ~]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-7-191 ~]# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:96498ffd522e70807ab6384a5c0485a79b9c7c08ca79ba08623edcad1054e62d
Status: Downloaded newer image for hello-world:latest
```

Validation Steps

- sudo systemctl status docker shows “active (running)”.
- docker run hello-world prints the “Hello from Docker!” confirmation message.

***Validation Screenshot** (Mandatory) —*



Explanation

This step installs the Docker Engine, which consists of a background daemon (dockerd) and the docker command-line client, giving the instance the ability to build, ship, and run containers. Installing via yum was chosen because Amazon Linux 2 provides a maintained docker package in its repositories, making this the simplest and most reliable installation path for a lab environment. Docker is an important foundation on this instance because it enables running isolated, reproducible application environments — including, later, Jenkins build agents or containerized versions of the other tools — without "works on my machine" inconsistencies.

Real-Time Use Case

In real organizations, Docker is installed on build and application servers so that applications can be packaged with all their dependencies into a single portable image. CI/CD pipelines in Jenkins commonly build a Docker image, run automated tests inside a container, and push the image to a registry (such as Docker Hub or Amazon ECR) as part of every deployment.

Common Errors and Troubleshooting

Error	Cause	Resolution
"docker: command not found"	Installation did not complete or repository was unreachable	Re-run sudo yum install docker -y and confirm internet/outbound connectivity
"Got permission denied while trying to connect to the Docker daemon socket"	ec2-user is not yet part of the docker group in the current shell session	Run newgrp docker, or log out and log back in via SSH, then retry
Docker service not running after instance reboot	Docker service was not enabled to start on boot	Run sudo systemctl enable docker

Task 3: Install Jenkins

Outcome

Jenkins is installed, running as a systemd service on port 8080, and accessible through a browser at http://<Public-IP>:8080, unlocked using the initial administrator password.

Objective

To set up a Continuous Integration / Continuous Deployment (CI/CD) automation server capable of building, testing, and deploying applications.

Prerequisites

- Task 1 completed — EC2 instance running and reachable.
- Port 8080 open in the security group (configured in Task 1, Step 3).
- Java (JDK) available, since Jenkins requires a supported Java runtime.

Step-by-Step Implementation

Step 1: Install Java

```
yum install java-openjdk21 -y
```

***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 ~]#  yum install java-17-amazon-corretto-devel -y
Last metadata expiration check: 0:08:40 ago on Mon Jul 13 10:14:20 2026.
Dependencies resolved.
================================================================================
 Package                         Arch   Version                    Repository   Size
================================================================================
Installing:
 java-17-amazon-corretto-devel   x86_64 1:17.0.19+10-1.amzn2023.1 amazonlinux 142 k
Installing dependencies:
 alsa-lib                        x86_64 1.2.7.2-1.amzn2023.0.3     amazonlinux 503 k
 cairo                           x86_64 1.18.0-4.amzn2023.0.3      amazonlinux 717 k
 dejavu-sans-fonts               noarch 2.37-16.amzn2023.0.2       amazonlinux 1.3 M
 dejavu-sans-mono-fonts          noarch 2.37-16.amzn2023.0.2       amazonlinux 467 k
 dejavu-serif-fonts              noarch 2.37-16.amzn2023.0.2       amazonlinux 1.0 M
 fontconfig                      x86_64 2.13.94-2.amzn2023.0.2     amazonlinux 273 k
 fonts-filesystem                noarch 1:2.0.5-12.amzn2023.0.2    amazonlinux 9.5 k
 freetype                        x86_64 2.13.2-5.amzn2023.0.2      amazonlinux 422 k
 google-noto-fonts-common        noarch 20240401-1.amzn2023.0.2    amazonlinux  17 k
 google-noto-sans-vf-fonts       noarch 20240401-1.amzn2023.0.2    amazonlinux 593 k
 graphite2                       x86_64 1.3.14-7.amzn2023.0.3      amazonlinux  97 k
 harfbuzz                        x86_64 7.0.0-2.amzn2023.0.2       amazonlinux 873 k
 java-17-amazon-corretto-headless
                                 x86_64 1:17.0.19+10-1.amzn2023.1 amazonlinux  91 M
 javapackages-filesystem         noarch 6.0.0-7.amzn2023.0.6       amazonlinux  12 k
 langpacks-core-font-en          noarch 3.0-21.amzn2023.0.4        amazonlinux  10 k
 libX11                          x86_64 1.8.10-2.amzn2023.0.1      amazonlinux 659 k
 libX11-common                   noarch 1.8.10-2.amzn2023.0.1      amazonlinux 147 k
 libXau                          x86_64 1.0.11-6.amzn2023.0.1      amazonlinux  33 k
 libXext                         x86_64 1.3.6-1.amzn2023.0.1       amazonlinux  42 k
 libXrender                      x86_64 0.9.11-6.amzn2023.0.1      amazonlinux  29 k
 libbrotli                       x86_64 1.0.9-4.amzn2023.0.2       amazonlinux 315 k
 libjpeg-turbo                   x86_64 2.1.4-2.amzn2023.0.5       amazonlinux 190 k
 libpng                          x86_64 2:1.6.37-10.amzn2023.0.13  amazonlinux 128 k
 libxcb                          x86_64 1.17.0-1.amzn2023.0.1      amazonlinux 235 k
 pixman                          x86_64 0.43.4-1.amzn2023.0.4      amazonlinux 296 k
 xml-common                      noarch 0.6.3-56.amzn2023.0.2      amazonlinux  32 k

Transaction Summary
================================================================================
Install  27 Packages

Total download size: 100 M
Installed size: 262 M
Downloading Packages:
(1/27): alsa-lib-1.2.7.2-1.amzn2023.0.3.x86_64.rpm  9.4 MB/s | 503 kB     00:00
(2/27): cairo-1.18.0-4.amzn2023.0.3.x86_64.rpm       11 MB/s | 717 kB     00:00
(3/27): dejavu-sans-fonts-2.37-16.amzn2023.0.2.noar  18 MB/s | 1.3 MB     00:00
```

```
  Verifying        : graphite2-1.3.14-7.amzn2023.0.3.x86_64                  11/27
  Verifying        : harfbuzz-7.0.0-2.amzn2023.0.2.x86_64                    12/27
  Verifying        : java-17-amazon-corretto-devel-1:17.0.19+10-1.amzn2023.  13/27
  Verifying        : java-17-amazon-corretto-headless-1:17.0.19+10-1.amzn20  14/27
  Verifying        : javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch     15/27
  Verifying        : langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch       16/27
  Verifying        : libX11-1.8.10-2.amzn2023.0.1.x86_64                     17/27
  Verifying        : libX11-common-1.8.10-2.amzn2023.0.1.noarch              18/27
  Verifying        : libXau-1.0.11-6.amzn2023.0.1.x86_64                     19/27
  Verifying        : libXext-1.3.6-1.amzn2023.0.1.x86_64                     20/27
  Verifying        : libXrender-0.9.11-6.amzn2023.0.1.x86_64                 21/27
  Verifying        : libbrotli-1.0.9-4.amzn2023.0.2.x86_64                   22/27
  Verifying        : libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64               23/27
  Verifying        : libpng-2:1.6.37-10.amzn2023.0.13.x86_64                 24/27
  Verifying        : libxcb-1.17.0-1.amzn2023.0.1.x86_64                     25/27
  Verifying        : pixman-0.43.4-1.amzn2023.0.4.x86_64                     26/27
  Verifying        : xml-common-0.6.3-56.amzn2023.0.2.noarch                 27/27

Installed:
  alsa-lib-1.2.7.2-1.amzn2023.0.3.x86_64
  cairo-1.18.0-4.amzn2023.0.3.x86_64
  dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch
  dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
  dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
  fontconfig-2.13.94-2.amzn2023.0.2.x86_64
  fonts-filesystem-1:2.0.5-12.amzn2023.0.2.noarch
  freetype-2.13.2-5.amzn2023.0.2.x86_64
  google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
  google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
  graphite2-1.3.14-7.amzn2023.0.3.x86_64
  harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
  java-17-amazon-corretto-devel-1:17.0.19+10-1.amzn2023.1.x86_64
  java-17-amazon-corretto-headless-1:17.0.19+10-1.amzn2023.1.x86_64
  javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch
  langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
  libX11-1.8.10-2.amzn2023.0.1.x86_64
  libX11-common-1.8.10-2.amzn2023.0.1.noarch
  libXau-1.0.11-6.amzn2023.0.1.x86_64
  libXext-1.3.6-1.amzn2023.0.1.x86_64
  libXrender-0.9.11-6.amzn2023.0.1.x86_64
  libbrotli-1.0.9-4.amzn2023.0.2.x86_64
  libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
  libpng-2:1.6.37-10.amzn2023.0.13.x86_64
  libxcb-1.17.0-1.amzn2023.0.1.x86_64
  pixman-0.43.4-1.amzn2023.0.4.x86_64
  xml-common-0.6.3-56.amzn2023.0.2.noarch

Complete!
```

Step 2: Add the Jenkins Repository

wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/rpm-stable/jenkins.repo

sudo yum install fontconfig java-21-openjdk

***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 ~]#  wget -O /etc/yum.repos.d/jenkins.repo \
    https://pkg.jenkins.io/rpm-stable/jenkins.repo
--2026-07-13 10:32:43--  https://pkg.jenkins.io/rpm-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 199.232.106.133, 2a04:4e42:5a::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|199.232.106.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jen 100%[===================>]     267  --.-KB/s    in 0s

2026-07-13 10:32:45 (8.03 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]
```

Step 3: Install Jenkins

```
yum install jenkins -y
```

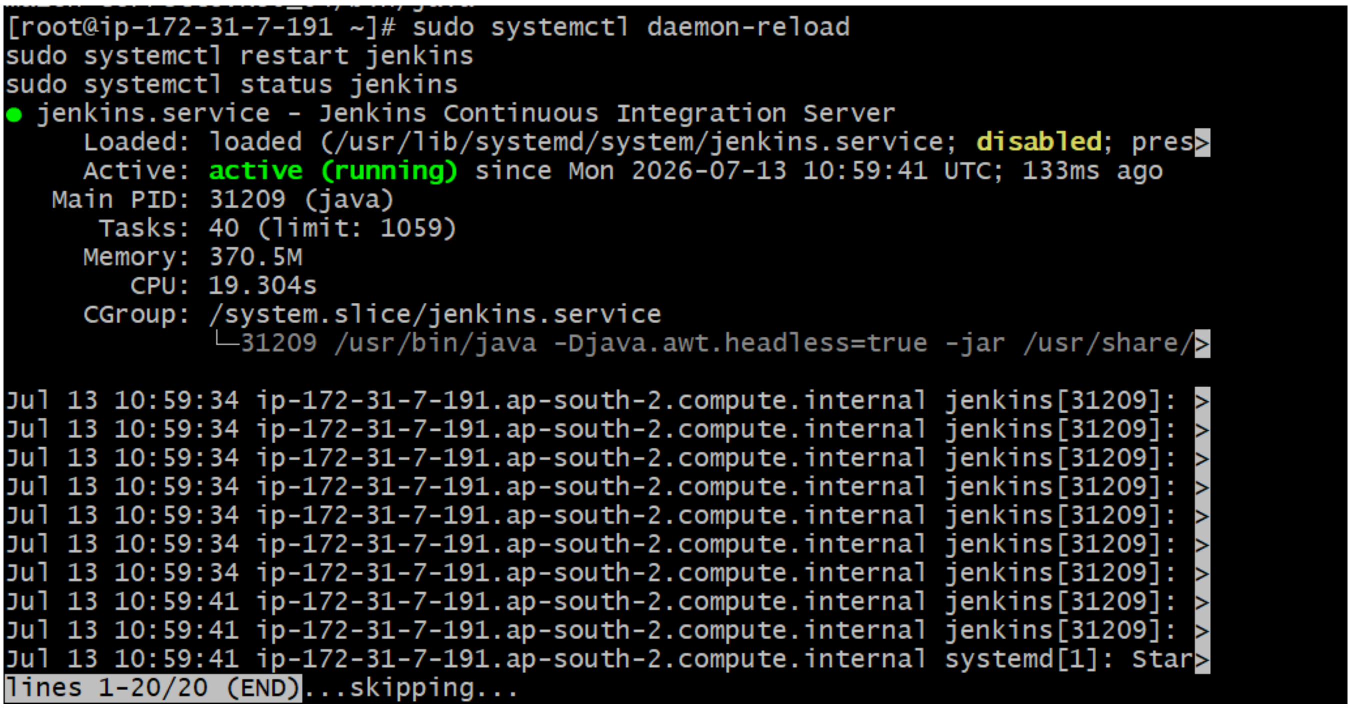
***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 ~]# yum install jenkins
Jenkins-stable                                  9.0 kB/s | 833  B     00:00
Jenkins-stable                                   95 kB/s | 1.6 kB     00:00
Importing GPG key 0x14ABFC68:
 Userid     : "Jenkins Project <jenkinsci-board@googlegroups.com>"
 Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
 From       : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Is this ok [y/N]: y
Jenkins-stable                                  2.4 kB/s | 2.5 kB     00:01
Last metadata expiration check: 0:00:01 ago on Mon Jul 13 10:33:17 2026.
Dependencies resolved.
================================================================================
 Package            Architecture      Version              Repository          Size
================================================================================
Installing:
 jenkins            noarch            2.568.1-1            jenkins             96 M

Transaction Summary
================================================================================
Install  1 Package

Total download size: 96 M
Installed size: 96 M
Is this ok [y/N]: y
Downloading Packages:
Jenkins-2.568.1-1.noarch.rpm                    9.4 MB/s |  96 MB     00:10
--------------------------------------------------------------------------------
Total                                           9.4 MB/s |  96 MB     00:10
Jenkins-stable                                  108 kB/s | 1.6 kB     00:00
Importing GPG key 0x14ABFC68:
 Userid     : "Jenkins Project <jenkinsci-board@googlegroups.com>"
 Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
 From       : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Is this ok [y/N]: y
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Running scriptlet: jenkins-2.568.1-1.noarch                               1/1
  Installing       : jenkins-2.568.1-1.noarch                               1/1
  Running scriptlet: jenkins-2.568.1-1.noarch                               1/1
  Verifying        : jenkins-2.568.1-1.noarch                               1/1

Installed:
  jenkins-2.568.1-1.noarch

Complete!
[root@ip-172-31-7-191 ~]#
```

Step 4: Start and Enable Jenkins

```
systemctl start jenkins

systemctl enable jenkins
systemctl status Jenkins
```

Screenshot Evidence *(Mandatory) —*



Step 5: Access Jenkins and Retrieve the Initial Admin Password

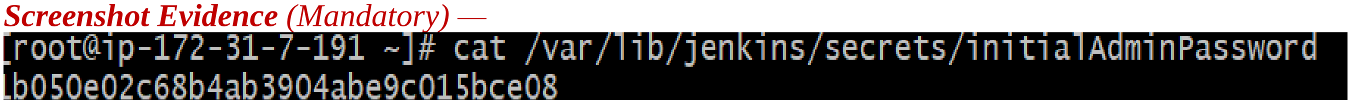
Open a browser and navigate to:

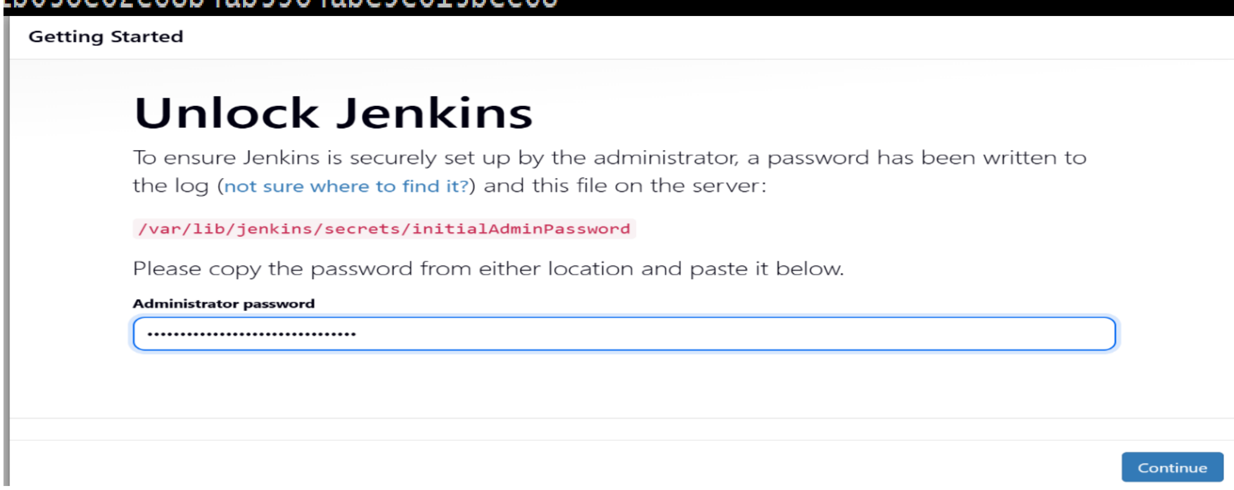
```
http://<Public-IP>:8080
```

Retrieve the password from the instance:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Screenshot Evidence *(Mandatory) —*

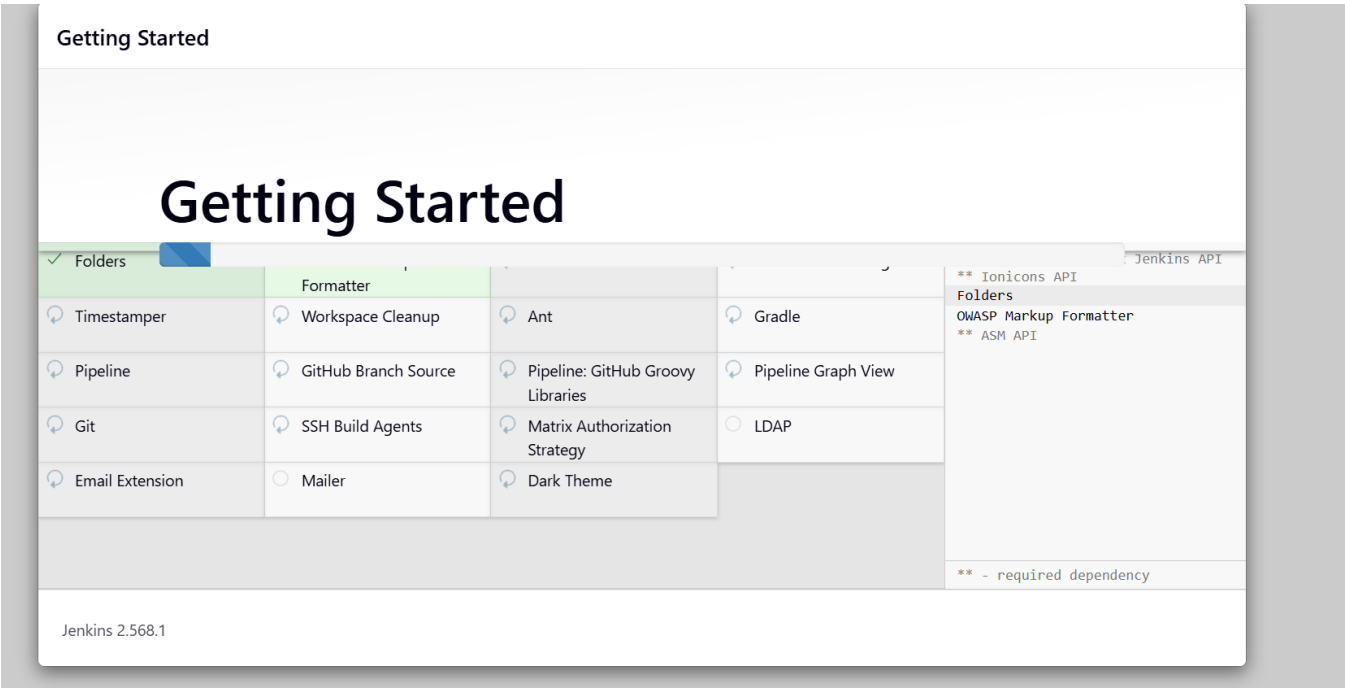


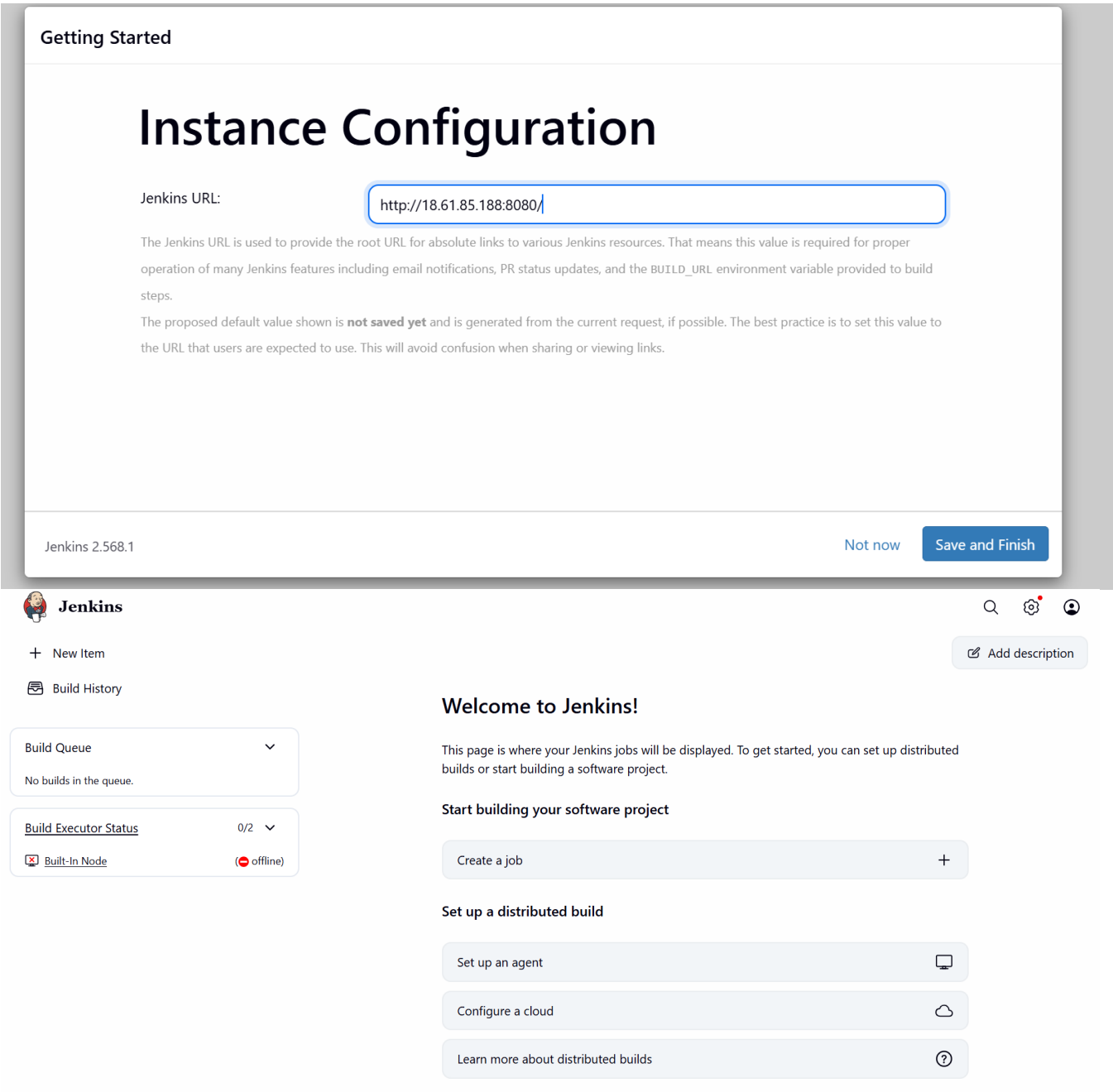


Step 6: Unlock Jenkins and Complete Setup

Paste the password on the “Unlock Jenkins” screen → click “Install suggested plugins” → create the first admin user → confirm the Jenkins URL → click “Start using Jenkins”.

***Screenshot Evidence** (Mandatory) —*





Validation Steps

- sudo systemctl status jenkins shows “active (running)”.
- The Jenkins dashboard loads successfully at http://<Public-IP>:8080 after setup.

Explanation

Jenkins is an open-source automation server that orchestrates the building, testing, and deployment of software through configurable pipelines and jobs. Java is installed first because Jenkins runs on the Java Virtual Machine and will fail to start without a compatible JDK. Jenkins is important on this instance because it represents the CI/CD control plane of the lab — in a real pipeline it would trigger builds, run tests, and could deploy artifacts to the Tomcat instance running alongside it.

Real-Time Use Case

Development teams use Jenkins to automatically build code on every commit, run unit and integration tests, and deploy the resulting artifact (for example, a WAR file) to an application server such as Tomcat, or build and push a Docker image as part of the same pipeline — all of which the tools on this single instance are capable of demonstrating together.

Common Errors and Troubleshooting

Error	Cause	Resolution
Browser cannot reach port 8080	Security group is missing the inbound rule for port 8080	Add an inbound TCP rule for port 8080 in the security group
Jenkins service fails to start	Java is not installed or an unsupported Java version is present	Install a supported JDK (e.g., java-openjdk11) and run sudo systemctl restart jenkins
initialAdminPassword file not found	Jenkins service has not finished starting up yet	Wait 30–60 seconds after starting the service, then re-run the cat command

Task 4: Install Apache (httpd)

Outcome

Apache HTTP Server (httpd) is installed and running on port 80, and its default (or a custom) test page is accessible at http://<Public-IP>.

Objective

To host a general-purpose web server on the instance capable of serving static websites or acting as a reverse proxy in front of other services.

Prerequisites

- Task 1 completed — EC2 instance running and reachable.
- Port 80 open in the security group (configured in Task 1, Step 3).

Step-by-Step Implementation

Step 1: Install Apache

```
yum install httpd -y
```

***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 ~]# yum install httpd -y
Last metadata expiration check: 0:48:44 ago on Mon Jul 13 10:33:17 2026.
Dependencies resolved.
================================================================================
 Package               Arch     Version                  Repository     Size
================================================================================
Installing:
 httpd                 x86_64   2.4.68-1.amzn2023.0.1    amazonlinux    46 k
Installing dependencies:
 apr                   x86_64   1.7.5-1.amzn2023.0.4     amazonlinux   129 k
 apr-util              x86_64   1.6.3-1.amzn2023.0.2     amazonlinux    97 k
 apr-util-lmdb         x86_64   1.6.3-1.amzn2023.0.2     amazonlinux    13 k
 generic-logos-httpd   noarch   18.0.0-12.amzn2023.0.3   amazonlinux    19 k
 httpd-core            x86_64   2.4.68-1.amzn2023.0.1    amazonlinux   1.4 M
 httpd-filesystem      noarch   2.4.68-1.amzn2023.0.1    amazonlinux    12 k
 httpd-tools           x86_64   2.4.68-1.amzn2023.0.1    amazonlinux    80 k
 mailcap               noarch   2.1.49-3.amzn2023.0.3    amazonlinux    33 k
Installing weak dependencies:
 apr-util-openssl      x86_64   1.6.3-1.amzn2023.0.2     amazonlinux    15 k
 mod_http2             x86_64   2.0.42-1.amzn2023.0.1    amazonlinux   167 k
 mod_lua               x86_64   2.4.68-1.amzn2023.0.1    amazonlinux    59 k

Transaction Summary
================================================================================
Install  12 Packages

Total download size: 2.0 M
Installed size: 6.2 M
Downloading Packages:
(1/12): apr-util-1.6.3-1.amzn2023.0.2.x86_64 2.6 MB/s |  97 kB     00:00
(2/12): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm  3.0 MB/s | 129 kB     00:00
(3/12): apr-util-lmdb-1.6.3-1.amzn2023.0.2.x 296 kB/s |  13 kB     00:00
(4/12): apr-util-openssl-1.6.3-1.amzn2023.0. 593 kB/s |  15 kB     00:00
(5/12): httpd-2.4.68-1.amzn2023.0.1.x86_64.r 2.0 MB/s |  46 kB     00:00
(6/12): generic-logos-httpd-18.0.0-12.amzn20 738 kB/s |  19 kB     00:00
(7/12): httpd-core-2.4.68-1.amzn2023.0.1.x86  43 MB/s | 1.4 MB     00:00
(8/12): httpd-filesystem-2.4.68-1.amzn2023.0 404 kB/s |  12 kB     00:00
(9/12): httpd-tools-2.4.68-1.amzn2023.0.1.x8 2.6 MB/s |  80 kB     00:00
(10/12): mailcap-2.1.49-3.amzn2023.0.3.noarc 1.2 MB/s |  33 kB     00:00
(11/12): mod_http2-2.0.42-1.amzn2023.0.1.x86 6.8 MB/s | 167 kB     00:00
(12/12): mod_lua-2.4.68-1.amzn2023.0.1.x86_6 2.1 MB/s |  59 kB     00:00
--------------------------------------------------------------------------------
Total                                        9.4 MB/s | 2.0 MB     00:00
Running transaction check
```

Step 2: Start and Enable Apache

```
systemctl start httpd
systemctl enable httpd

systemctl status httpd
```

***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 ~]# systemctl start httpd
[root@ip-172-31-7-191 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /
usr/lib/systemd/system/httpd.service.
[root@ip-172-31-7-191 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset:>
     Active: active (running) since Mon 2026-07-13 11:23:21 UTC; 28s ago
       Docs: man:httpd.service(8)
   Main PID: 32627 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; By>
      Tasks: 177 (limit: 1059)
     Memory: 15.0M
        CPU: 102ms
     CGroup: /system.slice/httpd.service
             ├─32627 /usr/sbin/httpd -DFOREGROUND
             ├─32645 /usr/sbin/httpd -DFOREGROUND
             ├─32647 /usr/sbin/httpd -DFOREGROUND
             ├─32648 /usr/sbin/httpd -DFOREGROUND
             └─32676 /usr/sbin/httpd -DFOREGROUND

Jul 13 11:23:21 ip-172-31-7-191.ap-south-2.compute.internal systemd[1]: Star>
Jul 13 11:23:21 ip-172-31-7-191.ap-south-2.compute.internal systemd[1]: Star>
Jul 13 11:23:21 ip-172-31-7-191.ap-south-2.compute.internal httpd[32627]: Se>
lines 1-19/19 (END)...skipping...
```

Step 3: Access Apache via Browser

```
http://<Public-IP>
```

***Screenshot Evidence** (Mandatory) —*

Validation Steps

- sudo systemctl status httpd shows “active (running)”.
- Browser displays the Apache test page or the custom message on port 80.

Explanation

Apache httpd is a widely used, modular HTTP server that serves web content and can also function as a reverse proxy or load balancer. It is installed via yum for simplicity and configured on its standard port 80 so that it behaves exactly as it would in a typical production deployment. Running Apache on this shared instance demonstrates how a general-purpose web server coexists with a CI server (Jenkins), a second web server (Nginx), and an application server (Tomcat) on the same host, each bound to its own port.

Real-Time Use Case

Apache is commonly used to serve static content, host simple websites, or sit in front of application servers as a reverse proxy that handles SSL termination, caching, and request routing before forwarding traffic to backend services such as Tomcat.

Common Errors and Troubleshooting

Error	Cause	Resolution
Browser shows “Connection refused” on port 80	Security group missing rule for port 80, or httpd service not started	Add inbound rule for port 80; run sudo systemctl start httpd
“Address already in use” when starting httpd	Another process is already bound to port 80	Identify the conflicting process with sudo ss -tulpn | grep :80 and stop it, or change Apache’s Listen port
Changes to index.html not visible	Browser cache showing an old page	Hard-refresh the browser (Ctrl+F5) or open in a private window

Task 5: Install Nginx

Outcome

Nginx is installed and running on port 8081 (changed from its default of port 80 to avoid conflicting with Apache), and its welcome page is accessible at http://<Public-IP>:8081.

Objective

To provide a lightweight, high-performance web server / reverse proxy alongside Apache, useful for practicing load balancing and reverse-proxy configurations.

Prerequisites

- Task 1 completed — EC2 instance running and reachable.
- Task 4 completed — Apache already occupies port 80, so Nginx must use a different port.
- Port 8081 open in the security group (configured in Task 1, Step 3).

Step-by-Step Implementation

Step 1: Install Nginx

```
yum install nginx -y
```

Screenshot Evidence *(Mandatory) —*

```
[root@ip-172-31-7-191 ~]# yum install nginx
Last metadata expiration check: 0:52:48 ago on Mon Jul 13 10:33:17 2026.
Dependencies resolved.
================================================================================
 Package            Arch     Version                     Repository     Size
================================================================================
Installing:
 nginx              x86_64   1:1.30.3-1.amzn2023.0.1     amazonlinux    34 k
Installing dependencies:
 gperftools-libs    x86_64   2.9.1-1.amzn2023.0.3        amazonlinux   308 k
 libunwind          x86_64   1.4.0-5.amzn2023.0.3        amazonlinux    66 k
 nginx-core         x86_64   1:1.30.3-1.amzn2023.0.1     amazonlinux   709 k
 nginx-filesystem   noarch   1:1.30.3-1.amzn2023.0.1     amazonlinux    10 k
 nginx-mimetypes    noarch   2.1.49-3.amzn2023.0.3       amazonlinux    21 k

Transaction Summary
================================================================================
Install  6 Packages

Total download size: 1.1 M
Installed size: 3.7 M
Is this ok [y/N]: y
Downloading Packages:
(1/6): gperftools-libs-2.9.1-1.amzn2023.0.3. 6.9 MB/s | 308 kB     00:00
(2/6): nginx-1.30.3-1.amzn2023.0.1.x86_64.rp 718 kB/s |  34 kB     00:00
(3/6): libunwind-1.4.0-5.amzn2023.0.3.x86_64 1.3 MB/s |  66 kB     00:00
(4/6): nginx-mimetypes-2.1.49-3.amzn2023.0.3 912 kB/s |  21 kB     00:00
(5/6): nginx-filesystem-1.30.3-1.amzn2023.0. 356 kB/s |  10 kB     00:00
(6/6): nginx-core-1.30.3-1.amzn2023.0.1.x86_  18 MB/s | 709 kB     00:00
--------------------------------------------------------------------------------
Total                                         7.4 MB/s | 1.1 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Running scriptlet: nginx-filesystem-1:1.30.3-1.amzn2023.0.1.noarch       1/6
  Installing       : nginx-filesystem-1:1.30.3-1.amzn2023.0.1.noarch       1/6
  Installing       : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch          2/6
  Installing       : libunwind-1.4.0-5.amzn2023.0.3.x86_64                 3/6
  Installing       : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64           4/6
  Installing       : nginx-core-1:1.30.3-1.amzn2023.0.1.x86_64             5/6
  Installing       : nginx-1:1.30.3-1.amzn2023.0.1.x86_64                  6/6
```

```
--------------------------------------------------------------------------------
Total                                           7.4 MB/s | 1.1 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Running scriptlet: nginx-filesystem-1:1.30.3-1.amzn2023.0.1.noarch        1/6
  Installing       : nginx-filesystem-1:1.30.3-1.amzn2023.0.1.noarch        1/6
  Installing       : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch           2/6
  Installing       : libunwind-1.4.0-5.amzn2023.0.3.x86_64                  3/6
  Installing       : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64            4/6
  Installing       : nginx-core-1:1.30.3-1.amzn2023.0.1.x86_64              5/6
  Installing       : nginx-1:1.30.3-1.amzn2023.0.1.x86_64                   6/6
  Running scriptlet: nginx-1:1.30.3-1.amzn2023.0.1.x86_64                   6/6
  Verifying        : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64            1/6
  Verifying        : libunwind-1.4.0-5.amzn2023.0.3.x86_64                  2/6
  Verifying        : nginx-1:1.30.3-1.amzn2023.0.1.x86_64                   3/6
  Verifying        : nginx-core-1:1.30.3-1.amzn2023.0.1.x86_64              4/6
  Verifying        : nginx-filesystem-1:1.30.3-1.amzn2023.0.1.noarch        5/6
  Verifying        : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch           6/6

Installed:
  gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
  libunwind-1.4.0-5.amzn2023.0.3.x86_64
  nginx-1:1.30.3-1.amzn2023.0.1.x86_64
  nginx-core-1:1.30.3-1.amzn2023.0.1.x86_64
  nginx-filesystem-1:1.30.3-1.amzn2023.0.1.noarch
  nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
```

Step 2: Change the Default Listening Port to 8081

Edit the main configuration file:

```
sudo vi /etc/nginx/nginx.conf
```

Locate the line listen 80 default_server; inside the server block and change it to:

```
listen 8081 default_server;
```

Save and exit (:wq).

***Screenshot Evidence** (Mandatory) —*

```
server {
    listen       81;
    listen       [::]:81;
    server_name  _;
    root         /usr/share/nginx/html;

    # Load configuration files for the default server block.
    include /etc/nginx/default.d/*.conf;

    error_page 404 /404.html;
    location = /404.html {
    }

    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
    }
}
```

Step 3: Start and Enable Nginx

```
systemctl start nginx
systemctl enable nginx

systemctl status nginx
```

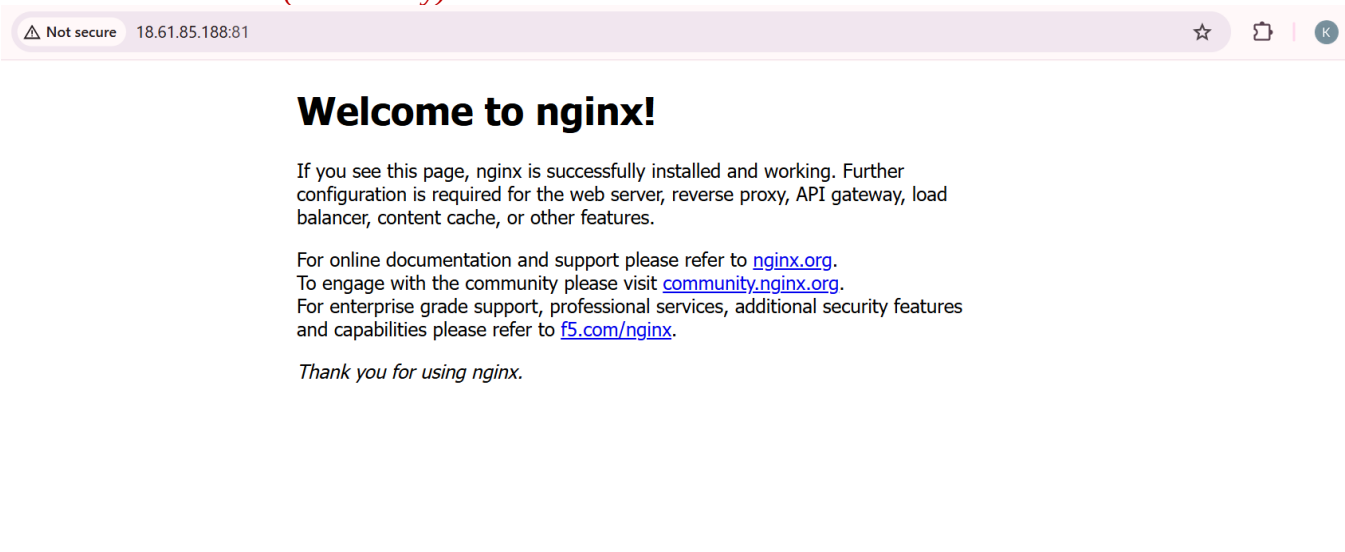
***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 ~]# systemctl start nginx
[root@ip-172-31-7-191 ~]# systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /
usr/lib/systemd/system/nginx.service.
[root@ip-172-31-7-191 ~]# systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset:>
     Active: active (running) since Mon 2026-07-13 11:28:48 UTC; 24s ago
   Main PID: 33728 (nginx)
      Tasks: 3 (limit: 1059)
     Memory: 4.1M
        CPU: 61ms
     CGroup: /system.slice/nginx.service
             ├─33728 "nginx: master process /usr/sbin/nginx"
             ├─33729 "nginx: worker process"
             └─33730 "nginx: worker process"

Jul 13 11:28:48 ip-172-31-7-191.ap-south-2.compute.internal systemd[1]: Star>
Jul 13 11:28:48 ip-172-31-7-191.ap-south-2.compute.internal nginx[33726]: ng>
Jul 13 11:28:48 ip-172-31-7-191.ap-south-2.compute.internal nginx[33726]: ng>
Jul 13 11:28:48 ip-172-31-7-191.ap-south-2.compute.internal systemd[1]: Star>
lines 1-16/16 (END)
```

Step 4: Access Nginx via Browser

```
http://<Public-IP>:8081
```

***Screenshot Evidence** (Mandatory) —*



Validation Steps

- sudo systemctl status nginx shows “active (running)”.
- Browser displays the Nginx welcome page (“Welcome to nginx!”) on port 8081.

Explanation

Nginx is a lightweight, event-driven web server and reverse proxy known for handling large numbers of concurrent connections efficiently. Its default listening port was changed from 80 to 8081 because Apache is already bound to port 80 on this same instance — this is a practical demonstration of how multiple web servers must be configured with distinct ports when co-located on a single host. Running both Apache and Nginx side by side highlights their different strengths and is a common pattern when comparing performance or gradually migrating traffic from one to the other.

Real-Time Use Case

Organizations often use Nginx as a reverse proxy or load balancer in front of multiple backend application servers, or run it alongside Apache during a migration, with Nginx handling static content and high-concurrency traffic while Apache or Tomcat handles dynamic content.

Common Errors and Troubleshooting

Error	Cause	Resolution
nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)	Apache (httpd) is already listening on port 80 on the same instance	Change Nginx’s listen directive to 8081 as shown in Step 2 before starting the service
Port 8081 not reachable from browser	Security group missing inbound rule for port 8081	Add an inbound TCP rule for port 8081
Configuration test fails after editing nginx.conf	Syntax error introduced while editing	Run sudo nginx -t to validate the configuration before restarting the service

Task 6: Install Apache Tomcat

Outcome

Apache Tomcat is installed under /opt/tomcat9 and running on port 8082 (changed from its default of 8080 to avoid conflicting with Jenkins), with its welcome page accessible at http://<Public-IP>:8082.

Objective

To provide a Java Servlet and JSP container capable of hosting Java web applications packaged as WAR files, completing the CI/CD path from Jenkins build to Tomcat deployment.

Prerequisites

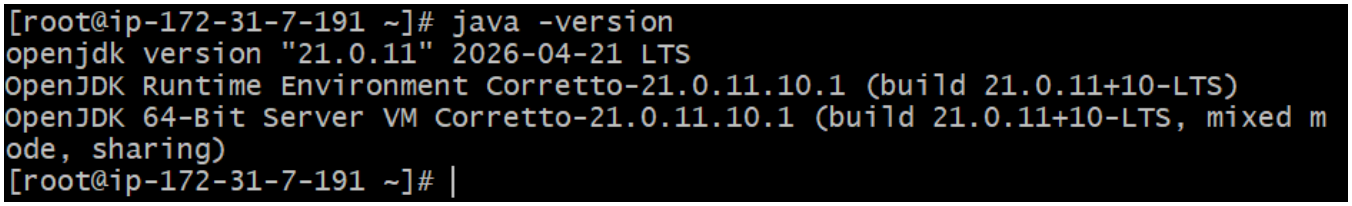
- Task 1 completed — EC2 instance running and reachable.
- Task 3 completed — Java already installed, and Jenkins already occupies port 8080, so Tomcat must use a different port.
- Port 8082 open in the security group (configured in Task 1, Step 3).

Step-by-Step Implementation

Step 1: Verify Java Is Available

```
java -version
```

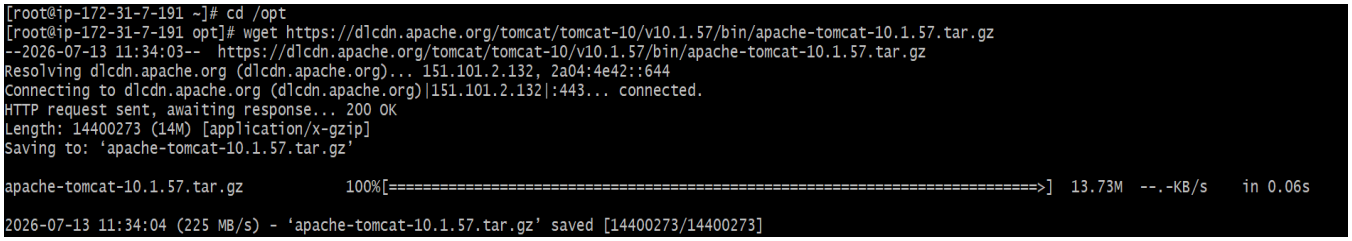
***Screenshot Evidence** (Mandatory) —*



Step 2: Download and Extract Tomcat

```
cd /opt
wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.57/bin/apache-tomcat-10.1.57.tar.gz
tar -xvzf apache-tomcat-10.1.57.tar.gz
mv apache-tomcat-10.1.57 tomcat10
```

***Screenshot Evidence** (Mandatory) —*



Step 3: Change the Default Connector Port to 8082

```
sudo vi /opt/tomcat9/conf/server.xml
```

Locate the line:

```
<Connector port="8080" protocol="HTTP/1.1" ... />
```

Change 8080 to 8082, then save and exit (:wq).

***Screenshot Evidence** (Mandatory) —*

```
<!-- A "Connector" represents an endpoint by which requests are received
     and responses are returned. Documentation at :
     HTTP Connector: /docs/config/http.html
     AJP  Connector: /docs/config/ajp.html
     Define a non-SSL/TLS HTTP/1.1 Connector on port 8080
-->
<Connector port="8081" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443"
           maxParameterCount="1000"
           />
<!-- A "Connector" using the shared thread pool-->
<!--
<Connector executor="tomcatThreadPool"
           port="8081" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443"
           maxParameterCount="1000"
           />
-->
```

Step 4: Set Permissions and Start Tomcat

```
sudo chmod +x /opt/tomcat9/bin/*.sh
sudo /opt/tomcat9/bin/startup.sh
```

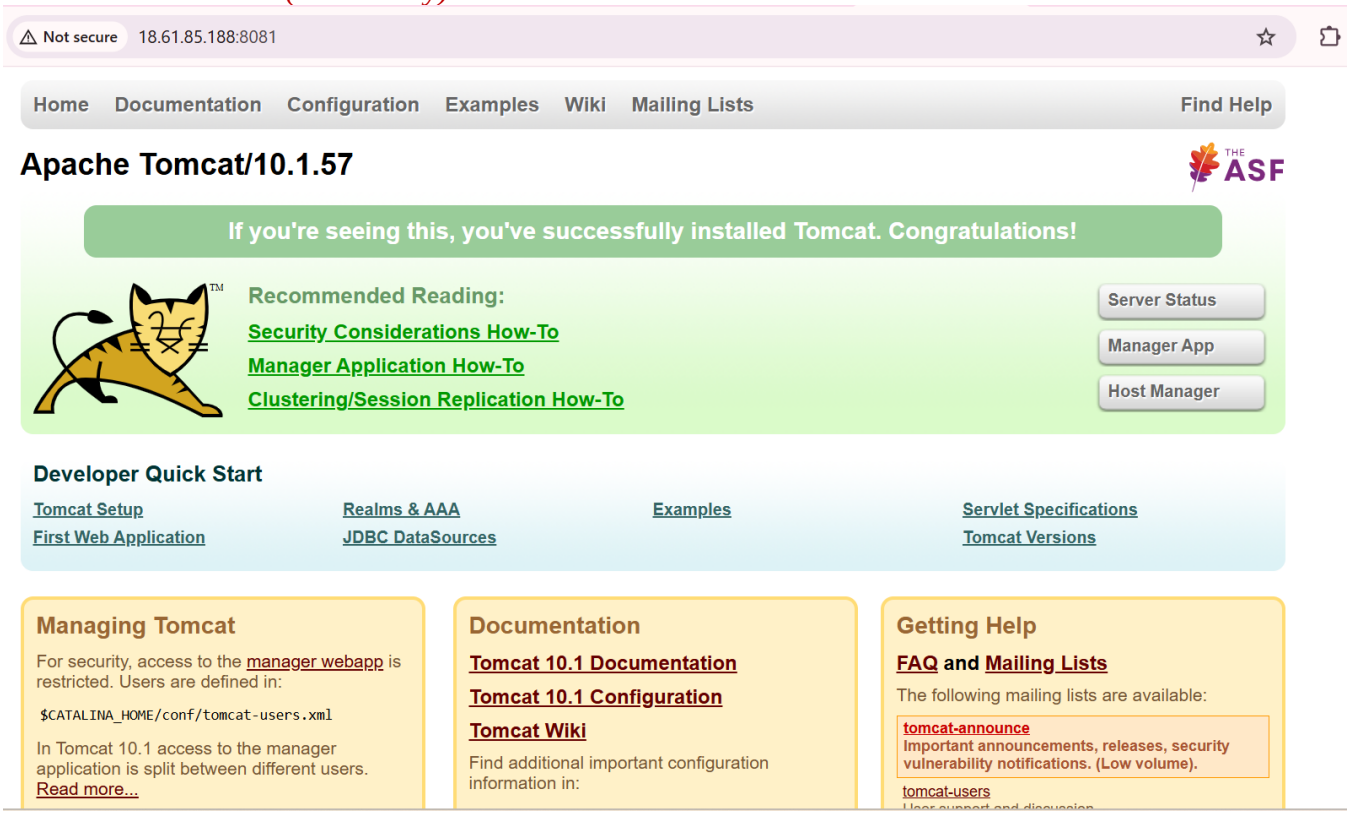
***Screenshot Evidence** (Mandatory) —*

```
[root@ip-172-31-7-191 opt]# mv apache-tomcat-10.1.57 tomcat10
[root@ip-172-31-7-191 opt]# vi /opt/tomcat10/conf/server.xml
```

Step 5: Access Tomcat via Browser

```
http://<Public-IP>:8082
```

***Screenshot Evidence** (Mandatory) —*



Validation Steps

- ps -ef | grep tomcat shows the Tomcat Java process running.
- Browser displays the Apache Tomcat welcome page on port 8082.**Explanation**

Apache Tomcat is a Java Servlet container that runs Java-based web applications (WAR files) and provides the HTTP layer, servlet engine (Catalina), and JSP engine needed to serve them. Its default Connector port was changed from 8080 to 8082 because Jenkins already occupies port 8080 on this same instance, again illustrating that co-located services must each be assigned a unique port. Tomcat completes the toolchain on this instance: in a full pipeline, Jenkins would build a Java application into a WAR file and deploy it directly into Tomcat's webapps directory.

Real-Time Use Case

Tomcat is widely used to host Java web applications and REST APIs in both development and production environments. In CI/CD pipelines, Jenkins is commonly configured to copy a freshly built WAR file into Tomcat's webapps folder (or call Tomcat's manager API) as the final "deploy" step, automating the path from source code commit to a running application.

Common Errors and Troubleshooting

Error	Cause	Resolution
Port 8080 conflict when starting Tomcat	Jenkins is already bound to port 8080 on the same instance	Change the Connector port in server.xml to 8082 as shown in Step 3 before starting Tomcat
"Permission denied" when running startup.sh	Shell scripts in the bin directory are not executable	Run sudo chmod +x /opt/tomcat9/bin/*.sh and retry
Tomcat welcome page does not load in the browser	Security group missing rule for port 8082, or Java is not installed/misconfigured	Add an inbound TCP rule for port 8082; confirm java -version returns a valid version

Final Verification: All Services Running Together

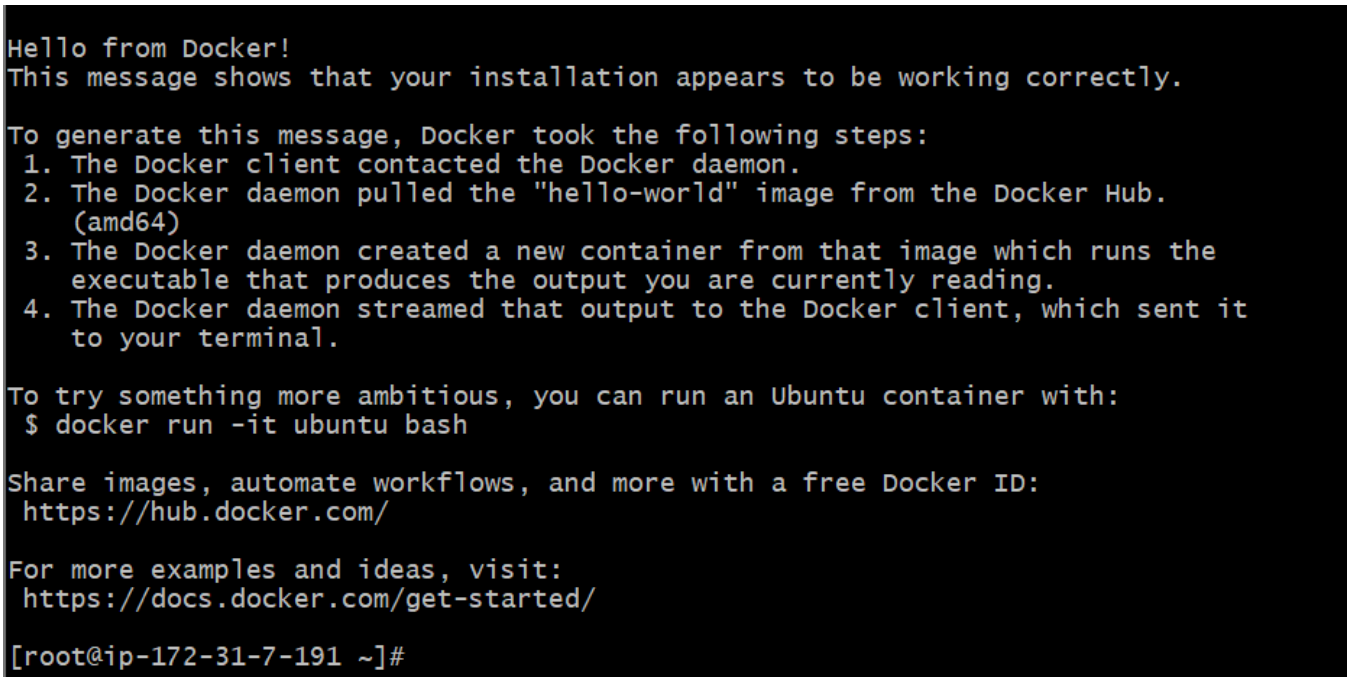
After completing all six tasks, the following command confirms that every service is active on the same EC2 instance:

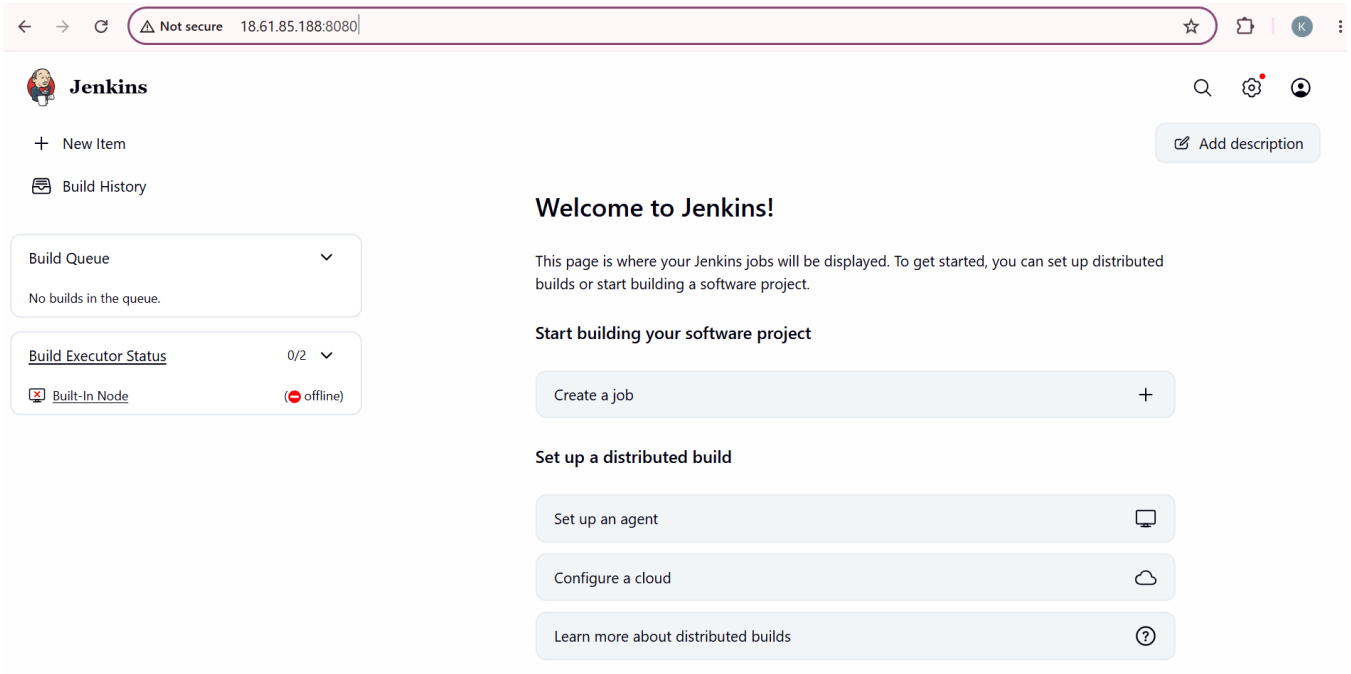
```
sudo systemctl status docker jenkins httpd nginx
```

Together with the running Tomcat process, this confirms all five tools are live simultaneously on one instance:

Service	Port	Verification Command / URL
Docker	N/A	docker run hello-world
Jenkins	8080	http://<Public-IP>:8080
Apache (httpd)	80	http://<Public-IP>
Nginx	8081	http://<Public-IP>:8081
Apache Tomcat	8082	http://<Public-IP>:8082

***Screenshot Evidence** (Mandatory) —*





It works!

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Home Documentation Configuration Examples Wiki Mailing Lists Find Help

Apache Tomcat/10.1.57

THE ASF

If you're seeing this, you've successfully installed Tomcat. Congratulations!

Recommended Reading:

Security Considerations How-To
Manager Application How-To
Clustering/Session Replication How-To

Server Status
Manager App
Host Manager

Developer Quick Start

Tomcat Setup
First Web Application
Realms & AAA
JDBC DataSources
Examples
Servlet Specifications
Tomcat Versions

Managing Tomcat

For security, access to the manager webapp is restricted. Users are defined in:

`$CATALINA_HOME/conf/tomcat-users.xml`

In Tomcat 10.1 access to the manager application is split between different users.
Read more...

Documentation

Tomcat 10.1 Documentation
Tomcat 10.1 Configuration
Tomcat Wiki

Find additional important configuration information in:

Getting Help

FAQ and Mailing Lists

The following mailing lists are available:

tomcat-announce
Important announcements, releases, security vulnerability notifications. (Low volume).

tomcat-users